

Research Proposal: Designing a Cognitively Accessible Coding Assistant with a Strategic User Interface and a Layer of Memory-Aware LLM Orchestration for Efficient Multi-Step Reasoning

Guadalupe Selene Vásquez Fabela
Parsons School of Design, The New School, New York, USA
Universidad Nacional Autónoma de México, Mexico City, Mexico
`vasql438@newschool.edu`

May 3, 2026

Abstract

Beginner programmers often experience cognitive friction when using chat-based large language model (LLM) coding assistants. Expanding conversation histories, repeated code regenerations, and over-generation of alternatives can overwhelm working memory, fragment attention, and obscure learning signals. These effects are especially pronounced for users with executive function variability, including individuals with ADHD (attention deficit and hyperactivity disorder), but also impact novice learners more broadly. I present a cognitively accessible coding assistant designed for beginner programmers (ages 18–35) learning through project-based building. The system integrates a strategic user interface with a memory-aware LLM orchestration layer to jointly reduce cognitive and computational overhead. The orchestration layer maintains a concise, state-aware representation of the evolving codebase, preventing redundant context accumulation and unnecessary regenerations. In parallel, the interface applies chunking, progressive disclosure, and constrained choice architecture to scaffold multi-step reasoning and reduce cognitive branching. I evaluate the system against a chat-based system with naive or unoptimized memory by measuring task completion, sustained attention, perceived workload, and token consumption growth dynamics. I hypothesize that memory-aware orchestration shifts token scaling from quadratic toward quasi-linear growth across multi-turn sessions, while interface constraints reduce cognitive overload during learning. Preliminary findings suggest that aligning computational efficiency with cognitively grounded interface design can reduce token waste, lower cognitive friction, and improve engagement and conceptual understanding in novice coding workflows.

1. Precedents and Introduction

Chat-based large language models (LLMs) are increasingly used as coding assistants, particularly by beginner programmers engaging in project-based learning. While these systems can accelerate development and reduce

syntactic overhead, they often introduce a different form of friction: expanding conversational histories, repeated full-code regenerations, and over-generation of alternatives that require evaluation and filtering. As multi-step tasks unfold, both the interaction transcript and the cognitive demands placed on the user grow.

This growth is not merely experiential but structural. In standard chat-based interfaces, prior conversational turns are re-injected into the model context at each interaction. As conversations lengthen, context size increases linearly per turn, while cumulative token consumption grows quadratically. This scaling behavior has computational implications, including increased latency and cost, but it also has cognitive consequences. Users must repeatedly parse expanding transcripts, maintain awareness of prior state, and determine which portions of generated code are relevant to current goals.

Working memory and executive function play a central role in managing such demands. Executive functions include high-order cognitive processes such as working memory, inhibitory control, cognitive flexibility, planning, and reasoning (Cristofori et al., 2019). When interaction structures require users to track long conversational histories, evaluate multiple regenerated outputs, and reconstruct evolving code state, these executive resources are taxed. This effect may be particularly salient for individuals with attention deficit and hyperactivity disorder (ADHD), who often experience variability in sustained attention and working memory capacity (Mildner et al., 2025; Schreiber et al., 2014). However, the underlying constraints apply broadly to novice learners navigating complex multi-step tasks. Existing research has examined LLM productivity gains (Brynjolfsson et al., 2023; Noy and Zhang, 2023), cognitive risks associated with over-reliance on AI systems (Mosier et al., 1996; Parasuraman and Riley, 2000), and strategies for optimizing token efficiency (Shazeer et al., 2017; Dao et al., 2022a). Yet these domains are typically treated separately. Little work formally connects the structural scaling behavior of conversational memory with user-level cognitive load in multi-step coding workflows. In particular, the relationship between token growth dynamics and executive function demands remains under-theorized.

I argue that computational efficiency and cognitive accessibility are structurally intertwined. When conversational memory grows without constraint, cumulative token scaling increases, regeneration frequency rises, and interaction fragmentation becomes more likely. Conversely, if task-relevant state can be serialized into a bounded, structured representation, both computational overhead and cognitive branching may be reduced.

To investigate this claim, I design a cognitively accessible coding assistant that integrates (1) a strategic user interface grounded in chunking, progressive disclosure, and constrained choice architecture, and (2) a memory-aware orchestration layer that maintains a concise, state-aware representation of the evolving codebase. I formally analyze token growth dynamics and empirically evaluate whether structured memory orchestration shifts cumulative scaling from quadratic toward quasi-linear growth, while improving task completion, comprehension, and perceived cognitive control.

By aligning computational scaling properties with principles of cognitive accessibility, this work positions memory-aware orchestration not merely as an engineering optimization, but as a design intervention aimed at supporting sustained, structured reasoning in novice coding environments.

2. Problem Statement and Research Goals

Chat-based LLM interfaces can introduce inefficiencies that impact both user cognition and system performance. In coding tasks, users often request full code regenerations or incomplete modifications, leading

to unnecessary token consumption and cognitive effort. Traditional chat memory strategies may fail to scale as conversations grow, sometimes resulting in quadratic token growth and context truncation. These inefficiencies disproportionately affect users with executive function challenges, such as individuals with ADHD, by increasing cognitive load. (Mildner et al., 2025; Schreiber et al., 2014; Cristofori et al., 2019). Another important factor is computational waste, which I define operationally as excess token consumption relative to the minimal tokens required to implement a functional requirement. In chat-based interfaces, two major contributors to waste are:

1. Quadratic growth of token usage as context expands, especially in multi-turn conversations (Vaswani et al., 2017; Dao et al., 2022b).
2. UX-induced regeneration, where users request repeated outputs due to friction in conversational structures that may not optimally convey information (e.g. prompting "give me the full code" even though only a few lines need to be regenerated).

Moreover, over-generation of content, such as requesting multiple alternative outputs or repeated regenerations, imposes an additional cognitive burden on users. Each new output must be evaluated, compared, and filtered, which increases executive function stress. This effect is particularly pronounced for individuals with attention deficit and hyperactivity disorder (ADHD), who often face challenges in sustaining attention, prioritizing information, and managing working memory. (Mildner et al., 2025). This is closely related to deficits in executive function, defined as the functions that "include high-order cognitive abilities such as working memory, inhibitory control, cognitive flexibility, planning, reasoning, and problem solving" Cristofori et al. (2019). As a result, inefficient interaction patterns not only waste computational resources but also amplify cognitive load, potentially reducing task performance and engagement.

Optimizing token consumption in chat-based coding environments is therefore crucial. Context-aware orchestration and memory-efficient strategies can mitigate quadratic growth, maintain relevant state, and reduce redundant API calls, ultimately lowering both computational costs and cognitive load (Beltagy et al., 2020; Dao et al., 2022b).

These approaches also support neurodivergent users who may struggle with managing extensive conversational histories, enabling a more focused and productive interaction with the AI.

3. Token Growth in Chat-Based LLM Interfaces

In standard chat-based LLM interfaces, prior conversational turns are re-injected into the context window at each new interaction. Let m_i denote the token length of message i . The total number of tokens sent to the model at turn t is:

$$C_t = \sum_{i=1}^t m_i \quad (1)$$

This represents full-history context injection.

Constant-Length Approximation

If each message has approximately constant length k , then:

$$C_t \approx kt \quad (2)$$

Thus, context length grows linearly with the number of turns.

Cumulative Token Consumption

Total tokens processed after n turns is the sum of all contexts:

$$T(n) = \sum_{t=1}^n C_t \quad (3)$$

Substituting $C_t \approx kt$:

$$T(n) = \sum_{t=1}^n kt \quad (4)$$

Using the closed-form expression for the arithmetic series:

$$T(n) = k \frac{n(n+1)}{2} \quad (5)$$

Asymptotically:

$$T(n) = O(n^2) \quad (6)$$

Therefore, although context growth per turn is linear, cumulative token consumption grows quadratically.

Sliding Window Strategy

If only the last w messages are injected:

$$C_t = \sum_{i=t-w}^t m_i \quad (7)$$

Under constant message length:

$$C_t \approx kw \quad (8)$$

Which implies:

$$T(n) = O(n) \quad (9)$$

This reduces computational growth but sacrifices long-term conversational continuity.

Structured Memory Orchestration

If only relevant structured state s_t is injected at each turn:

Token Growth in Chat-Based LLM Interfaces

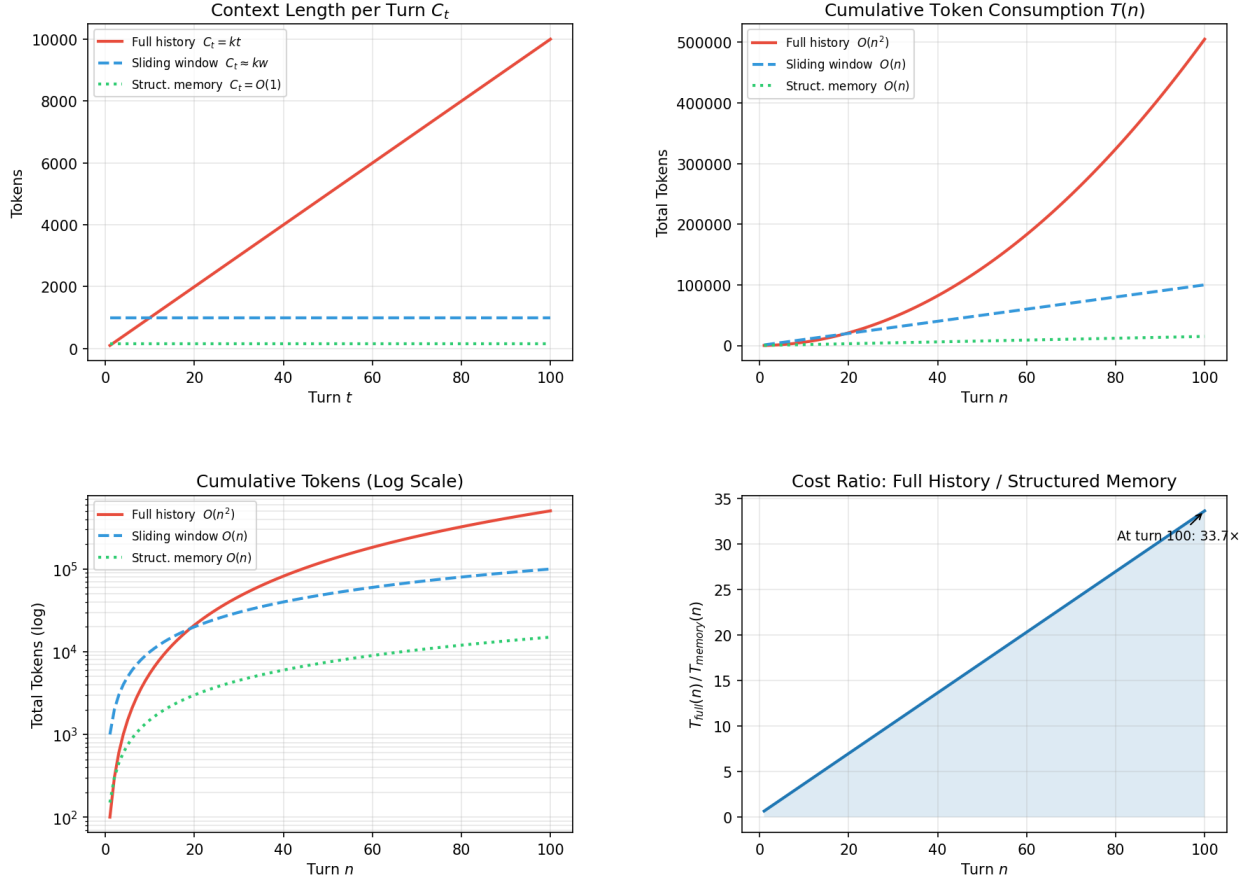


Figure 1: Comparison of quadratic token growth in naive chat memory (or considering the full history of the chat, versus quasi-linear growth under sliding window and structured memory orchestration.

$$C_t = m_t + s_t \quad (10)$$

If s_t grows sublinearly or remains bounded:

$$C_t = O(1) \quad (11)$$

Which implies:

$$T(n) = O(n) \quad (12)$$

This motivates memory-aware orchestration strategies that preserve task state while preventing quadratic growth. In my proposed system, the context, or the last state of the codebase, is also visible to the user through the user interface, and adjusted before reaching the LLM to

My goal is to design and evaluate a cognitively accessible coding assistant that:

- Allows for the user to visualize the context sent to the LLM.

- Minimizes token consumption using a memory-aware orchestration layer.
- Supports multi-step coding tasks while reducing cognitive load.
- Enhances engagement through interface design guided by UX principles.

3.1 Research Questions

I want to compare the two user interfaces, a chat-based interface, and my structured memory orchestration (SMO) system.

RQ1: Does SMO reduce cumulative token growth across multi-step coding tasks?

RQ2: Does orchestration reduce completion time and regeneration frequency?

RQ3: Does orchestration improve code comprehension, and perceived control over the workload?

3.2 Hypotheses

H1: Cumulative token consumption will grow subquadratically in the orchestrated (SMO) condition compared to the chat condition.

H2: Participants will complete tasks faster in the orchestrated condition.

H3: Participants will score higher on code comprehension in the orchestrated condition.

H4: Bounded memory growth and structured state visibility reduce interaction fragmentation, which may manifest as increased user engagement and sustained task focus across turns.

4. Proposed System

The web application includes a system of a strategic user interface (UI) and an orchestration layer, functioning as a “working memory”.

Technology Stack: HTML/CSS/JS, CodeMirror, p5.editor, ChatGPT API, SQLite.

4.1 Strategic UI

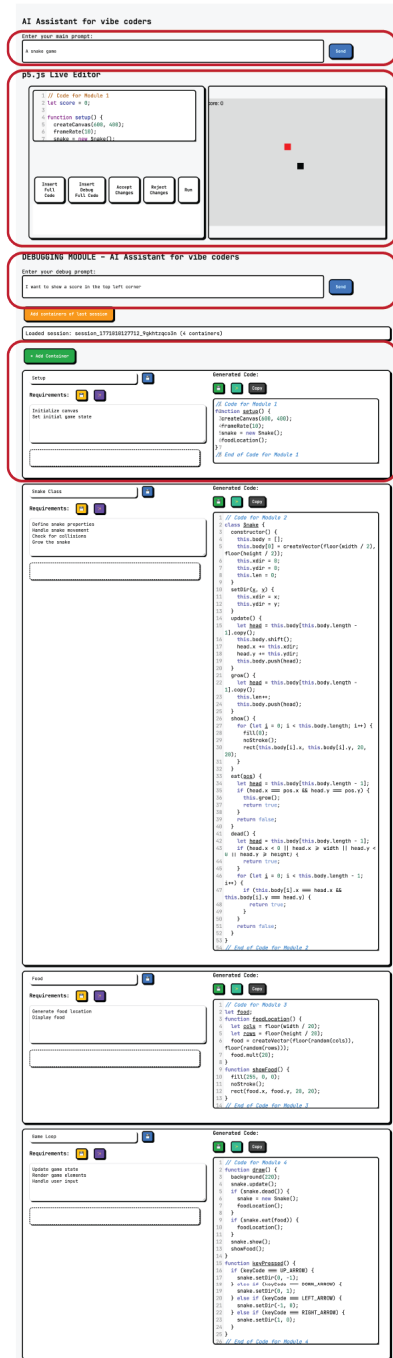
The elements of the UI are shown in Figure 2 and Figure 3.

e1. Prompt textbox: Sends the user prompt and initial context to the API in order to get the preliminary code in the compiler e2, or a first sample of code that, if approved, will fill the containers (e4).

e2. Compiler: Shows and compiles the code. The user can directly edit the code. It can also receive code from the prompt textbox or from containers (e4).

e3. Debugging textbox: Selectively takes context from e2 and e4. When the user asks for a modification, the API response is first tested in the compiler e2 before being incorporated into containers e4.

USER INTERFACE (1/2)



e1. Prompt textbox

Enter your main prompt:

e2. Compiler

p5.js Live Editor

```
1 // Code for Module 1
2 let score = 0;
3
4 function setup() {
5   createCanvas(600, 400);
6   frameRate(10);
7   snake = new Snake();
8 }
```

Score: 0

e3. Debugging textbox

DEBUGGING MODULE - AI Assistant for vibe coders

Enter your debug prompt:

e4. Containers

+ Add Container

Setup

Requirements:

Initialize canvas
Set initial game state

Generated Code:

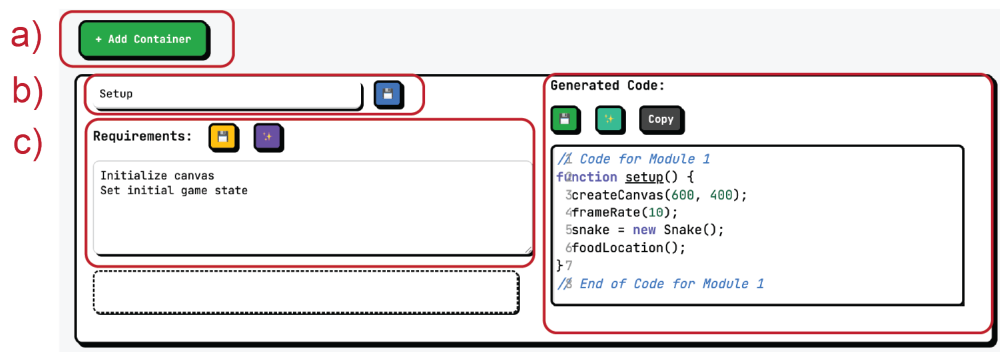
```
// Code for Module 1
function setup() {
  createCanvas(600, 400);
  frameRate(10);
  snake = new Snake();
  foodLocation();
}
// End of Code for Module 1
```

Individual container

Figure 2: Strategic UI — Diagram 1

USER INTERFACE (2/2)

e4. Containers - Close up



- a) “Add container” button
- b) Title textbox
- c) Requirements textbox
- d) Code textbox

Figure 3: Strategic UI — Diagram 2

e4. Containers: Each session will have multiple containers that are either generated automatically from the compiler e2 once the preliminary code is accepted, or manually:

- Automatically generated from the compiler once preliminary code is accepted
- Manually created via:
 - a) **“Add container” button:** it adds a new container.
 - b) **Title textbox:** the user can insert a title and generate requirements and code based on that.
 - c) **Requirements textbox:** the user can manually insert the requirements and save them, or generate them with AI as a function of the title of the container.
 - d) **Code textbox:** here is the code that responds to the requirements. It can either be AI generated as a function of the requirements, or the user can insert its own code, or a hybrid.

4.2 Orchestration Layer

The dynamics of the orchestration layer are shown in Figures 4, 5, and 6, as a function of the components of the UI.

4.3 Considerations

Design Translation of Cognitive Principles

The proposed system translates these cognitive constraints into concrete interface and orchestration decisions:

- **Chunking.** Code and requirements are segmented into containers, reducing the number of active elements users must track simultaneously.
- **Modularity.** Each container encapsulates a specific functional requirement, limiting branching and aligning with Hick’s Law by reducing simultaneous choice sets.
- **Progressive Disclosure.** Only relevant context and editable sections are surfaced at a given time, mitigating working memory overload.
- **Low-friction Interaction.** The interface is designed to scaffold executive function by making modifications feel incremental and bounded rather than global and disruptive.
- **Strategic LLM Orchestration.** Instead of repeatedly regenerating full conversational history, the system injects only structured, task-relevant state representations. This reduces unnecessary regeneration, lowers token waste, and minimizes cognitive branching caused by evaluating redundant outputs.

Baseline Memory Behavior

For comparison, the baseline chat interface employs a sliding context window that retains up to the 20 most recent messages in a conversation. Because the experimental tasks are expected to be completed in fewer than 20 interaction turns, the baseline condition behaves equivalently to naive full conversational replay within this range. That is, for $0 \leq t \leq 20$, context size grows approximately linearly with turn count:

ORCHESTRATION LAYER (1/3)

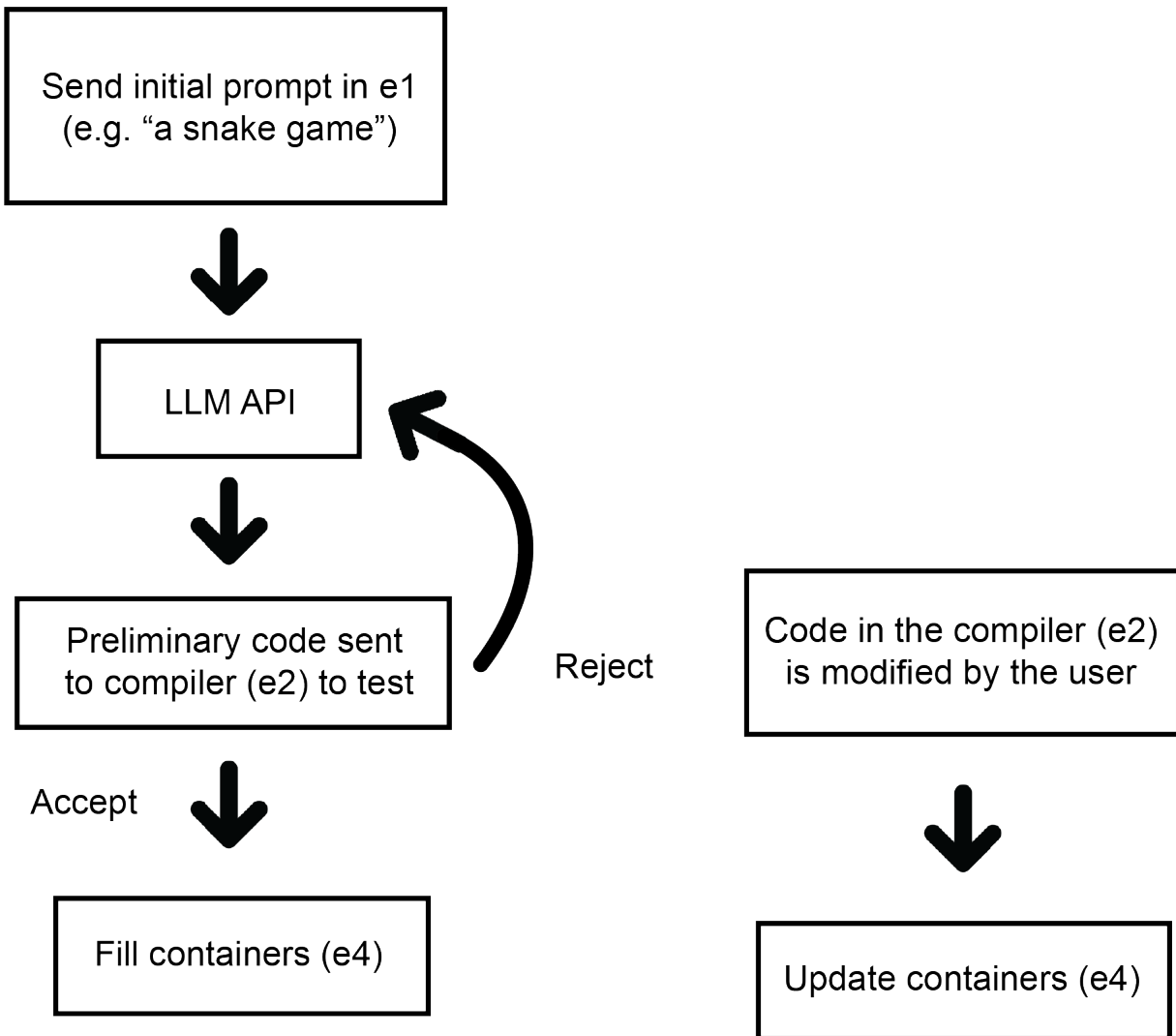


Figure 4: Orchestration Layer — Context Routing Logic

ORCHESTRATION LAYER (2/3)

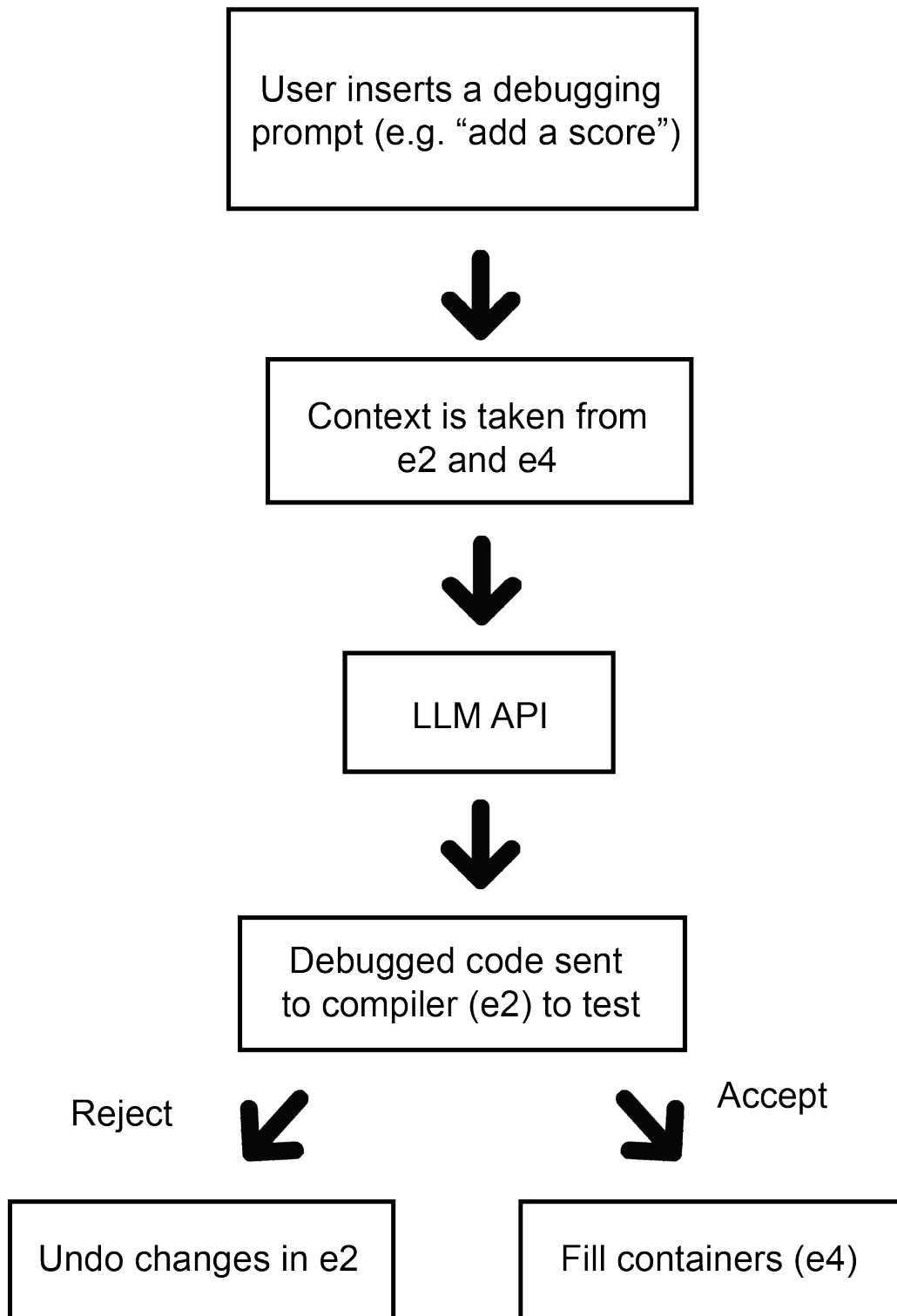


Figure 5: Orchestration Layer — Memory Optimization Flow

ORCHESTRATION LAYER (3/3)

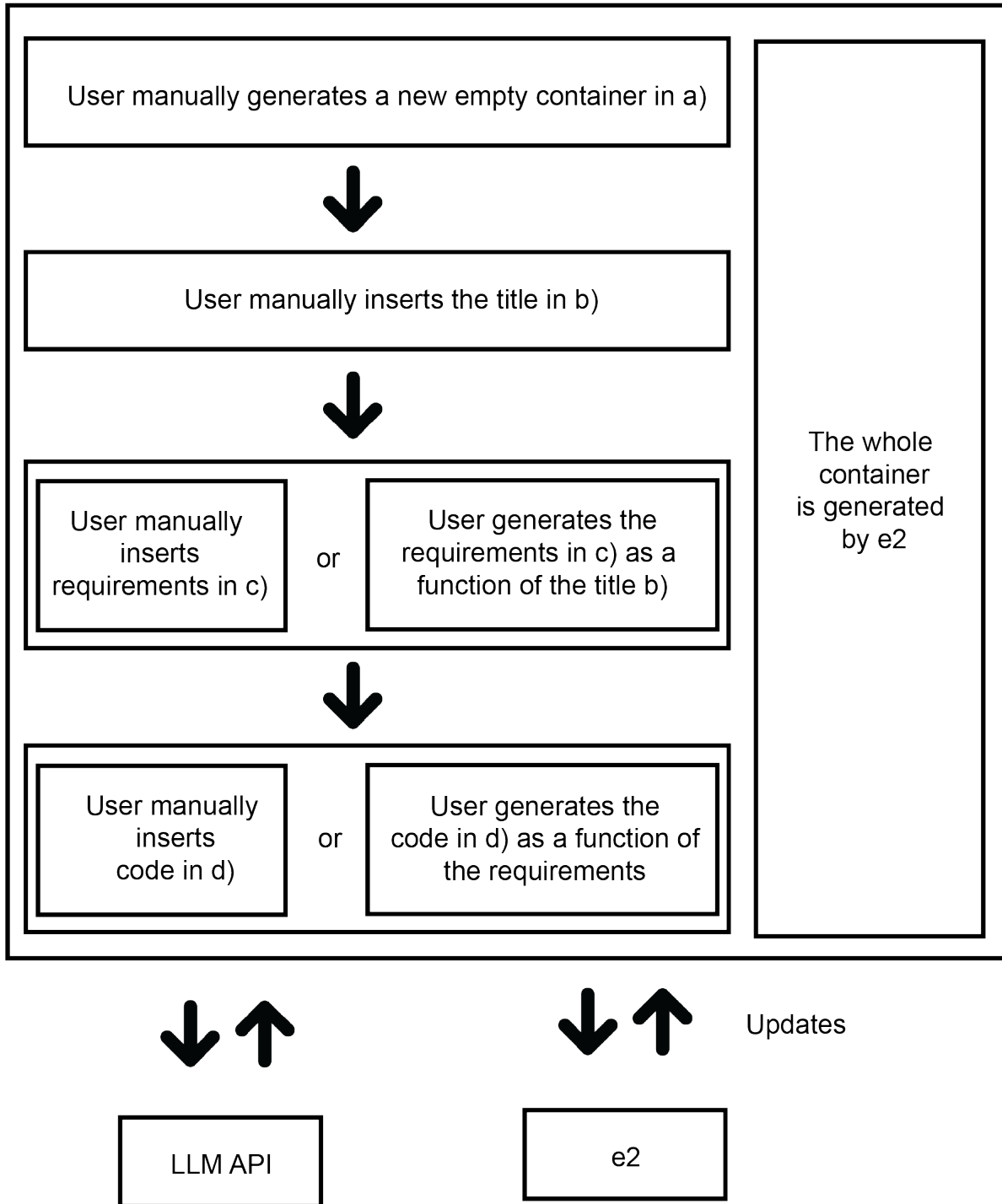


Figure 6: Orchestration Layer — Token Reduction Dynamics

$$C_t = \sum_{i=1}^t m_i$$

Under this regime, cumulative token consumption exhibits quadratic growth:

$$T(n) = O(n^2).$$

Thus, within the expected task horizon, the baseline condition preserves full conversational accumulation, allowing direct comparison against the structured memory orchestration (SMO) condition.

5. Methodology

This study employs a counterbalanced within-subjects experimental design to evaluate the impact of interface-level memory orchestration on LLM-assisted coding performance.

Two conditions are compared:

1. **Baseline Chat Interface:** A traditional chat-based interface with a sliding context window injecting up to the 20 most recent messages. (In tasks with less than 20 messages, this behaves exactly as a naive conversational accumulation, or unoptimized memory)
2. **Orchestrated Interface:** The proposed cognitively accessible interface incorporating a strategic orchestration layer for memory optimization and token reduction across multi-step tasks.

Both conditions use the same underlying LLM model (`chatgpt-4o-mini`) to isolate the effect of interface design and orchestration strategy. Although the model is not state-of-the-art, performance is intentionally held constant across conditions to ensure that any observed differences are attributable to interface structure and memory management rather than model capability.

5.1 Participants

I will recruit a statistically significant sample of young and early-middle-age adults (18–35) who self-report prior experience using AI-assisted coding tools.

Participants will include individuals who self-report an ADHD diagnosis to explore potential interaction effects between cognitive profile and interface design.

Inclusion Criteria:

- Prior experience using AI coding assistants (e.g., Copilot, ChatGPT, Claude)
- Basic familiarity with JavaScript

Prior proficiency in JavaScript and `p5.js` will be assessed via a short pre-study questionnaire to control for baseline skill variation.

5.2 Task

Participants will implement a functional Snake game in JavaScript using `p5.js`, assisted by the LLM interface assigned in each condition (chat-based, and the strategic user interface).

The tasks were selected because they:

- Require multi-step reasoning
- Involve iterative debugging and modification
- Necessitate maintaining structured state across multiple prompts
- Produce objectively verifiable functional outcomes

The implementation must satisfy the following functional requirements:

- The snake begins as a single square centered on the canvas.
- Movement is possible in four directions (up, down, left, right).
- The snake grows by one segment upon colliding with food.
- Food disappears upon collision and respawns at a random location.
- Each food instance appears in a different random color.
- A score counter increments by one per food collision.
- Collision with walls or self triggers game over.
- A restart mechanism resets score and game state.

Each participant completes the task twice (once per user interface). Order is counterbalanced to mitigate learning effects (first chat-based, then SMO interface, and viceversa). Another group will be asked to develop two different games: a snake game and a tetris game to observe biases.

After each task, participants complete a structured written questionnaire assessing subjective experience and comprehension.

6. Data Collection and Analysis

6.1 Automated Logging System

All LLM interactions are captured via an automated logging pipeline. For each session and interaction turn, the following metadata are recorded:

- Input token count
- Output token count
- Total token consumption

- Timestamp
- Session identifier

Token consumption is defined according to the model’s native tokenization algorithm and serves as the primary unit of computational efficiency.

Data are collected directly from the API to eliminate manual logging errors.

Token usage will be analyzed at three granularities:

- Per turn
- Per session
- Per successfully implemented functional requirement

6.2 Measures

Computational Efficiency

Efficiency is operationalized as:

- Total tokens consumed per session
- Tokens per turn
- Tokens per functional requirement achieved

Task Completion Time

Completion time (in minutes) is measured from first prompt submission to functional task completion. This metric captures interaction friction and orchestration effectiveness.

Subjective Evaluation

Participants complete Likert-scale and open-ended questionnaires assessing:

- Perceived cognitive load
- Perceived contextual clarity
- Satisfaction with interaction
- Perceived control over memory and instructions
- Perceived distraction or fragmentation

Code Comprehension

To assess depth of cognitive engagement, participants answer structured comprehension questions, such as:

- Where is collision detection implemented?
- How would you modify the food color logic?
- How would you increase movement speed?

Responses are evaluated using a rubric assessing:

- Correctness
- Specificity
- Evidence of structural understanding

6.3 Analysis Plan

Quantitative measures (token usage and completion time) will be analyzed using paired statistical tests (e.g., paired t-tests or nonparametric equivalents depending on distribution).

Effect sizes will be reported to evaluate practical significance.

I will additionally conduct mixed-effects analyses to explore interaction effects between interface condition and self-reported ADHD status as an exploratory group.

Qualitative responses will be coded using thematic analysis to identify recurring patterns in perceived clarity, control, and cognitive burden.

Statistical significance will be evaluated at $\alpha = 0.05$.

Metrics

6.4 Growth Structure Analysis

To evaluate the theoretical scaling claims of the proposed orchestration layer, I analyze token growth dynamics at two levels: (i) per-turn context size and (ii) cumulative token growth. These analyses directly test whether structured memory constrains context expansion relative to naive conversational accumulation.

6.4.1 Per-Turn Context Size Growth

Let C_t denote the number of tokens injected into the model context at interaction turn t . For each session:

$$C_t = \text{input tokens}_t + \text{structured state}_t$$

where $\text{structured state}_t$ captures any retrieved or serialized memory representations added to the prompt at turn t .

I compute the average per-turn context size:

$$\bar{C} = \frac{1}{n} \sum_{t=1}^n C_t$$

where n is the total number of turns in a session.

To assess boundedness of context growth, I additionally estimate the per-turn growth slope by fitting:

$$C_t = \beta_0 + \beta_1 t + \epsilon_t$$

A significantly smaller β_1 in the orchestrated condition relative to baseline chat would indicate attenuated context expansion.

Statistical comparisons are conducted using linear mixed-effects models with condition as a fixed effect and participant as a random intercept. When appropriate, random slopes for turn index are included.

6.4.2 Cumulative Token Scaling

Let $T(n)$ denote cumulative token consumption after n interaction turns:

$$T(n) = \sum_{t=1}^n C_t$$

Under naive chat memory injection (full conversational replay), cumulative growth is theoretically quadratic:

$$T(n) \sim \alpha n^2$$

Under structured orchestration with bounded retrieval, cumulative growth is expected to be approximately linear:

$$T(n) \sim \beta n$$

To empirically estimate the scaling behavior, I fit a power-law model:

$$T(n) = \gamma n^\kappa$$

Taking logarithms yields:

$$\log T(n) = \log \gamma + \kappa \log n$$

I estimate the scaling exponent κ via linear regression on log–log transformed data.

The primary parameter of interest is κ :

- $\kappa \approx 2$ indicates quadratic growth (naive conversational accumulation),
- $\kappa \approx 1$ indicates linear growth (bounded structured orchestration).

Exponents are compared across conditions using paired statistical tests or mixed-effects modeling. A statistically significant reduction in κ in the orchestrated condition would directly support the claim that structured memory bounds cumulative token growth.

6.5 Primary Outcome Measures

Primary metrics directly test efficiency and scaling behavior:

- **Cumulative token growth curve** $T(n)$
- **Per-turn growth slope** β_1
- **Scaling exponent** κ
- **Regeneration rate** (number of model re-queries per task)
- **Tokens per requirement** (tokens consumed per completed functional requirement)
- **Regeneration rate per requirement** (including full code rewrites)

6.6 Secondary Outcome Measures

Secondary metrics assess task efficiency and user cognition:

- **Task completion time**
- **Post-task comprehension assessment**
- **Qualitative interaction analysis**

6.7 Behavioral Engagement Metrics

These measures characterize interaction structure and agency:

- Number of voluntary refinements
- Ratio of user-written to AI-generated code
- Average uninterrupted user coding streak length
- Regeneration clustering (burstiness of repeated model calls)

6.8 Temporal Flow Analysis

Temporal metrics quantify interaction rhythm:

- Inter-prompt interval (time between user turns)
- Pause duration preceding regeneration
- Turn-to-turn latency variance

7. Expected Contributions

- A formal analysis of cumulative token growth in multi-step LLM interactions.
- A memory-aware orchestration architecture that bounds context growth while preserving task state.
- Empirical evidence that structured memory reduces computational waste in LLM-assisted workflows
- Interaction level findings on how memory visibility affects regeneration behavior.

8. Ethical Considerations

This study involves human participants interacting with AI-assisted coding interfaces and requires careful consideration of ethical principles in research design, data collection, and reporting. The following measures have been implemented to ensure compliance with ethical standards:

Informed Consent

All participants will provide informed consent prior to participation. Consent forms clearly describe:

- The purpose of the study, including the evaluation of LLM-assisted coding interfaces.
- Expected participant activities, time commitment, and potential risks.
- The voluntary nature of participation, with the right to withdraw at any time without penalty.
- Contact information for the research team for questions or concerns.

Consent language is written in accessible terms to accommodate diverse cognitive profiles, including individuals with executive function variability.

Data Privacy and Storage

Participant data include both behavioral metrics (e.g., token counts, interaction timestamps, code submissions) and self-reported information (e.g., ADHD status, experience with coding tools). To protect privacy:

- All personally identifiable information (PII) is anonymized and assigned unique participant codes.
- Interaction logs are stored on encrypted servers with restricted access to the research team.
- Data retention follows institutional guidelines, with secure deletion scheduled after the conclusion of the study and analysis.
- Aggregated results will be reported without identifying individual participants.

Risk Assessment

Potential risks to participants are minimal, but include:

- Cognitive fatigue from extended coding sessions or multi-step reasoning tasks.
- Possible frustration or stress when interacting with AI systems that generate incomplete or unexpected outputs.

Mitigation strategies:

- Tasks are designed to be manageable in duration (approximately 30–60 minutes per session).
- Breaks are allowed between tasks to reduce cognitive load.
- Participants may pause or discontinue participation at any time without consequence.
- Support is provided for participants who experience discomfort or confusion during the study.

Inclusion of Neurodivergent Participants

Special attention is given to neurodivergent participants (e.g., those with ADHD):

- Interfaces are designed to reduce cognitive overload, following principles such as chunking, progressive disclosure, and bounded choice architecture.
- The study ensures accessibility in instructions, task pacing, and interaction logging to avoid disadvantaging participants with executive function variability.

Institutional Review Board (IRB) Approval

The study protocol, including recruitment procedures, consent forms, data collection methods, and risk mitigation strategies, has been submitted to and approved by the Institutional Review Board (IRB) of The New School. Any modifications to the protocol that could impact participant safety or data privacy will be submitted for IRB review prior to implementation.

Ethical Data Use and Reporting

- All analyses and publications will aggregate results and avoid disclosing individual-level data.
- Any secondary use of collected data will require additional consent or IRB approval.
- The research team is committed to transparency, responsible reporting, and adherence to ethical standards for human subjects research.

9. Timeline

The project has a timeline of 7 months, and I am currently conducting the pilot study in Month 4.

Phase	Duration
Literature Review	Month 1
System Development	Months 2–3
Pilot Study	Month 4
Main Study	Month 5
Analysis	Month 6
Writing	Month 7

Table 1: Projected Research Timeline

10. Bibliography

References

- Beltagy, I., Peters, M. E., and Cohan, A. (2020). Longformer: The long-document transformer. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP 2020)*, pages 874–891.
- Brynjolfsson, E., Li, D., and Raymond, L. R. (2023). Generative AI at work. *Quarterly Journal of Economics*, 139(1):1–60.
- Cristofori, I., Cohen-Zimmerman, S., and Grafman, J. (2019). Executive functions. In J. Grafman and V. R. S. H. Salazar, editors, *Handbook of Clinical Neurology*, volume 163, pages 197–219. Elsevier.
- Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Ré, C. (2022a). Flashattention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS 2022)*, volume 35, pages 16344–16359.
- Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Ré, C. (2022b). Flashattention: Fast and memory-efficient exact attention with io-awareness. *arXiv preprint arXiv:2205.14135*. Discusses that standard self-attention has time and memory complexity quadratic in sequence length.
- Mildner, T., Fidel, D., Stefanidi, E., Woźniak, P. W., Malaka, R., and Niess, J. (2025). A comparative study of how people with and without ADHD recognise and avoid dark patterns on social media. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*, pages Article 222, 1–17, New York, NY, USA. ACM.
- Mosier, K. L., Skitka, L. J., Heers, S., and Burdick, M. (1996). Automation bias: Decision making and performance in high-technology cockpits. *The International Journal of Aviation Psychology*, 6(1):47–63.
- Noy, S. and Zhang, W. (2023). Experimental evidence on the productivity effects of generative artificial intelligence. *Science*, 381(6654):187–192.
- Parasuraman, R. and Riley, V. (2000). Humans and automation: Use, misuse, disuse, abuse. *Human Factors*, 39(2):230–253.

- Schreiber, J. E., Possin, K. L., Girard, J. M., and Rey-Casserly, C. (2014). Executive function in children with attention deficit/hyperactivity disorder: The NIH EXAMINER battery. *Journal of the International Neuropsychological Society*, 20(1):41–51.
- Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., and Dean, J. (2017). Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *Proceedings of the International Conference on Learning Representations (ICLR 2017)*.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS 2017)*, pages 5998–6008. Introduces the transformer architecture and discusses that self-attention has $O(n^2)$ complexity in sequence length.